MyCodeSchool

Adnotări Spring

Alex · Cafenea, pe etape

Etapa 1 · Componente și bean-uri

Adnotare	Ce face	În Cafenea
`@SpringBootApplication`	Clasa de pornire: pornește auto-configurarea și caută componente în pachetul ei.	`CafeneaApplication` pornește aplicația; meniul pornește din `run()` (`CommandLineRunner`).
`@ComponentScan("app")`	Pachetele în care Spring caută componente.	Mută `CafeneaApplication` în `app.pornire`: nu mai găsește `ViewCafenea`. Repară cu `@ComponentScan`.
`@Component`	Spring creează un singur obiect din clasă (bean) și îl dă oricui îl cere în constructor.	`ViewCafenea` și toate clasele `*Impl`. Clasele `*Singleton` se șterg.
`@Configuration`	Clasă în care descrii de mână cum se creează anumite obiecte.	`AppConfig`.
`@Bean`	Metodă din @Configuration: obiectul întors devine bean.	`Scanner` și `CalculatorPret` cu TVA 0.21.
`@Import`	Aduce în aplicație o clasă @Configuration pe care scanarea n-o vede.	`AppConfig` stă în pachetul `config`, în afara lui `app`.

Etapa 2 · Roluri

Adnotare	Ce face	În Cafenea
`@Service`	Un @Component pentru stratul de logică.	`ProdusServiceImpl`, `ComandaServiceImpl`.
`@Repository`	Un @Component pentru stratul de date.	`ProdusRepositoryImpl`, `ComandaRepositoryImpl`.

Etapa 3 · Configurare din fișier

Adnotare	Ce face	În Cafenea
`@Value("${cheie}")`	Injectează o valoare din configurare (`application.yml`, `.properties`).	`cafenea.tva=0.21`. `CalculatorPret` devine @Component; `@Bean` rămâne doar pe `Scanner`.
`@ConfigurationProperties`	Leagă proprietățile cu un prefix de câmpurile unui obiect.	`CafeneaProperties`: `nume`, `moneda`, `program`, folosite în meniu și pe bon.
`@EnableConfigurationProperties`	Activează o clasă @ConfigurationProperties.	Pe `AppConfig`, pentru `CafeneaProperties`.
`@PropertySource`	Încarcă un fișier de proprietăți în plus.	`program.properties` cu `cafenea.program=08:00–20:00`, afișat în antetul meniului.

Etapa 4 · Mai multe bean-uri de același tip

Adnotare	Ce face	În Cafenea
`@Autowired`	Arată ce constructor folosește Spring când sunt mai mulți.	Al doilea constructor în `ViewCafenea`, fără bonul detaliat. Marchează-l pe cel complet.
`@Primary`	Bean-ul ales implicit când sunt mai mulți de același tip.	`BonPrinter` simplu: bonul de la opțiunea 3.
`@Qualifier("nume")`	Cere un bean anume, după nume.	`BonPrinter` detaliat: opțiunea nouă 6, cu fiecare reducere pe rând.
`@Order(n)`	Ordinea bean-urilor primite într-o List: mai mic = primul.	Reduceri: întâi meniu cafea + croissant –2 lei, apoi fidelitate –5% peste 30 lei.

Etapa 5 · Câte obiecte și când se creează

Adnotare	Ce face	În Cafenea
`@Scope("singleton")`	Implicit: un singur obiect pentru toată aplicația.	`ComandaRepositoryImpl` își numără instanțele; raportul afișează 1.
`@Scope("prototype")`	Un obiect nou la fiecare cerere.	`Comanda` devine bean; fiecare comandă nouă vine din `ObjectProvider`.
`@Lazy`	Bean-ul se creează la prima folosire, nu la pornire.	`RaportVanzari` (opțiunea 5) anunță când e creat: abia la prima folosire.
`@DependsOn("nume")`	Forțează crearea altui bean înaintea acestuia.	`JurnalCafenea` pornește înaintea lui `ProdusRepositoryImpl`, fără să-i fie injectat.

Etapa 5T · Teste

Adnotare	Ce face	În Cafenea
`@Test`	Metodă rulată de JUnit ca test; pică la primul `assert` neîndeplinit.	Cele 9 teste: TVA 7.35, total 42.35, ordinea reducerilor, serviciul.
`@BeforeEach`	Rulează înaintea fiecărui test: fiecare test pornește de la obiecte noi.	`ComandaServiceImplTest`: serviciul nou, cu mock-urile, la fiecare test.
`@ExtendWith(MockitoExtension.class)`	Lasă Mockito să creeze câmpurile `@Mock` înainte de fiecare test, fără Spring.	Pe `ComandaServiceImplTest`: test unitar, contextul nu pornește.
`@Mock`	Obiect fals al unui tip: întoarce ce i se spune și ține minte apelurile.	Cele 4 dependențe ale lui `ComandaServiceImpl`, date prin constructor, cu `new`.
`@SpringBootTest`	Pornește contextul Spring întreg, ca la `mvn spring-boot:run`.	`CafeneaContextTest`: `ComandaService` real, cu toate bean-urile lui. Pornește și `CommandLineRunner`-ul.
`@MockitoBean`	Înlocuiește în context bean-ul de tipul câmpului cu un mock.	Pe `ViewCafenea` (fără el testul stă la `Optiune:`) și pe `ProdusRepository` (Espresso 8.00, total 9.68).

`when(...).thenReturn(...)`, `verify(...)`, `never()` și `ArgumentCaptor` nu sunt adnotări, ci metode și clase Mockito, folosite în corpul testului.

Etapa 6 · Ciclul de viață

Adnotare	Ce face	În Cafenea
`@PostConstruct`	Rulează o dată, după ce bean-ul a primit dependențele.	`ProdusRepositoryImpl` încarcă produsele din `produse.txt`.
`@PreDestroy`	Rulează o dată, la oprirea aplicației.	`ComandaRepositoryImpl` salvează comenzile în `comenzi.txt`.
`@Bean(initMethod, destroyMethod)`	Același lucru, pentru clase pe care nu le poți adnota.	`CasaDeMarcat` (pachetul `casa`, fără adnotări): `deschide()` și `inchide()` cu totalul zilei.

Etapa 7 · Condiții și profiluri

Adnotare	Ce face	În Cafenea
`@Profile("dev")`	Bean-ul există doar când profilul e activ.	Meniu de test cu 2 produse la `--spring.profiles.active=dev`.
`@ConditionalOnProperty`	Bean-ul există doar dacă o proprietate are o anumită valoare.	Reducerile pornesc doar cu `cafenea.reduceri.activ=true`.
`@ConditionalOnMissingBean`	Bean-ul se creează doar dacă nu există deja unul de același tip.	`Notificator` implicit pe consolă; dispare când adaugi alt `Notificator`.

Etapa 8 · Evenimente

Adnotare	Ce face	În Cafenea
`@EventListener`	Metoda rulează când e publicat evenimentul primit ca parametru.	`ComandaFinalizataEvent` → `Bucatarie` și `Statistici`. „Cafeneaua e deschisă” la `ApplicationStartedEvent`.

Etapa 9 · Fire și programare în timp

Adnotare	Ce face	În Cafenea
`@EnableAsync + @Async`	Metoda rulează pe alt fir; apelantul nu o așteaptă.	`Bucatarie` pregătește comanda 2 s pe alt fir; meniul merge mai departe.
`@EnableScheduling + @Scheduled`	Metoda rulează periodic.	Raport de vânzări la fiecare `cafenea.raport.ms`.

`CommandLineRunner` nu e adnotare, ci o interfață: metoda ei `run()` se execută după ce Spring a creat toate bean-urile.